Fruit Vision

CNN-Based Fruit Classification System

SOFTWARE DESIGN DOCUMENT



The Islamia University of Bahawalpur

RUBINA ANWAR **23405555017**

BASEERAT ARIF **23405555025**

[SESSION 2023-2025 BSCS]

DEPARTMENT OF COMPUTER SCIENCE & IT,

MTB COLLEGE SADIQABAD

Revision History

Date	Description	Author	Revision No.
22/11/2025	Version 1.0	Rubina Anwar	First Submission
25/02/2026	Version 1.1	Baseerat Arif	Second Submission

Document Approval

The following Software Requirements Specification has been accepted and approved by the following

Signature	Printed Name	Title	Date
	Mr. Hafiz Muhammad Naveed	Supervisor	

DECLARATION

I hereby declare that the project titled "**Fruit Vision**" is a record of original project work undertaken by me for the award of the degree Bachelors of Computer Science at MTB College, Khalid Campus, Sadiqabad, during the year 2023-25. I have completed this project under the guidance of **Mr. Hafiz Muhammad Naveed**.

I also declare that this project report has not been submitted previously for the award of any degree, diploma, associateship, fellowship, or any other title at any institution.

The Islamia University of Bahawalpur

MTB COLLEGE, KHALID CAMPUS, SADIQABAD
DEPARTMENT OF COMPUTER SCIENCE AND IT

CERTIFICATE

This is to certify that the Project work entitled **"Fruit Vision"** is a Bonafide work carried out by **Miss Rubina Anwar (23405555017), Baseerat Arif (23405555025)** in partial fulfillment of the requirements for the award of degree of **Bachelor of Computer Science** from **The Islamia University of Bahawalpur** during the year 2023-25.

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated into this report. The project report has been approved as it satisfies the academic requirements prescribed for the Bachelor's degree project work.

Signature of the Supervisor	Signature of the HOD
Mr. Hafiz Muhammad Naveed	Mr. Muhammad Umer

Acknowledgements

I am grateful to the Almighty **Allah**, who blessed me with health, wisdom, knowledge, thoughts and opportunity to make a contribution through this effort. I offer my humblest thanks from the deepest core of my heart to the **Holy Prophet Muhammad (Peace Be Upon Him)**, the most perfect and exalted among all who have ever lived on the face of the earth.

The research work embodied in this dissertation was accomplished under the able guidance and kind supervision of **Mr. Hafiz Muhammad Naveed, Lecturer**. I will always remember his moral encouragement, skillful guidance, positive criticism and valuable advice throughout the course of my study.

I also express my deep love and respect for my beloved parents, who are the greatest asset of my life and have always given me confidence and support. I am equally thankful to my friends, who deserve appreciation for their sincere encouragement, prayers, and support for my success.

Table of Contents

Chapter 1
Project Overview 1
1.1 Project Introduction 1
1.2 Background and Motivation 1
1.3 Objective of the Project 1
1.4 Features 2
1.4.2 Deep Learning Model 2
1.4.3 Real-Time Prediction 2
1.4.4 Web-Based Interface 2
1.4.5 Scalable and Extensible Design 2
1.5 Problem Statement 3
1.6 Scope of the Project 3
1.7 Expected Results 3
Chapter 2
Literature Review 4
2.1 Image Classification in Modern Applications 4
2.2 Current Fruit Classification Systems 4
2.3 Deep Learning in Image Recognition 6
2.4 Relevant Technologies for Fruit Classification 6
2.4.4 Fast API Framework 7
2.4.5 Transfer Learning and Fine-Tuning 7
2.5 Challenges in Fruit Classification 7
2.6 Research Gaps and Opportunities 7
2.7 Summary 8
Chapter 3 System Analysis 9
3.2 Functional Requirements 9
Working: 10
Importance: 10
3.2.2 Fruit Classification 10
Working: 10
Importance: 10
3.2.3 Real-Time Prediction 10
Importance: 11
3.2.4 Model Training Pipeline 11
Working: 11
3.2.5 User Interface 11
Features of UI: 11
Importance: 11
3.3 Non-Functional Requirements 12
Importance: 12
3.3.2 Performance 12
Importance: 12
3.3.3 Scalability 12
Importance: 12
3.3.4 Security 13
3.3.5 Usability 13
Importance: 13
3.3.6 Maintainability 13
Importance: 13

3.3.7 Portability 13
Importance: 13
3.3.8 Availability 14
3.4 System Components and Architecture 14
3.4.1 Hardware Components 14
3.4.2 Software Components 14
3.4.3 System Architecture 15
Presentation Layer: 15
Key Features of Presentation Layer: 15
Application Layer: 15
Key Features of Application Layer: 16
Intelligence Layer: 16
Key Features of Intelligence Layer: 16
3.5 System Interactions and Workflow 16
3.6 Performance Analysis 17
Below is a thorough examination of these standards: 17

Chapter 4 19

Requirement Specifications 19
1. Hardware Requirements 19
1.1 Software Requirements 20
1.2 System Constraints 21
1.3 User Requirements 22
1.4 Methodology for CNN-Based Fruit Classification System 22

Chapter 5 25

System Design 25
1.1 System Design Overview 25
1.2 Software Design 28
1.3 Sequence Diagram 30
1.4 User Interface Design 31

Chapter 6 32
System Context 32
6.1 Introduction 32
6.2 System Context Overview 32
6.3 Context Diagram 32
6.4 External Entities and Data Flows 33
6.5 Operating Environment 33
6.6 Assumptions and Dependencies 33

Chapter 7 34
Implementation 34
7.1 Implementation Overview 34
7.2 Technology Stack 34
7.3 Project Structure 34
7.4 Module Implementation 35
7.5 Running the Application 37
7.6 Working Application 37

Chapter 8 39
Testing 39
8.1 Testing Overview 39
8.2 Testing Strategy 39
8.3 Test Environment 39
8.4 Test Cases 39
8.5 Test Execution Results 40
8.6 Test Summary 41

Chapter 1

Project Overview

1.1 Project Introduction

The increasing demand for intelligent automation in agriculture and food processing has made AI-based classification systems essential for modern applications. For instance, manual fruit sorting is time-consuming, error-prone, and difficult to scale in commercial settings. This project presents Fruit Vision -- a CNN-Based Fruit Classification System using TensorFlow and Karas. To enable accurate and efficient fruit identification, the system combines a custom Convolutional Neural Network (CNN) architecture with a Fast API backend and an interactive web interface. Users can upload fruit images and receive real-time classification predictions. This improves sorting accuracy and operational efficiency in addition to convenience. The system is a feasible step toward fully automated produce classification because it is modular, easy to use, and scalable.

1.2 Background and Motivation

Quality assessment and wastage are common challenges in today's fast-paced food supply chains; relying solely on manual inspection poses consistency risks, and misclassification of produce raises costs. The project's objective is to develop an AI-enabled system that provides automated classification, real-time prediction, and accuracy improvements. The system guarantees efficiency and reliability by allowing users to classify fruit images through the use of deep learning technology with TensorFlow and Keras. This project shows how CNNs can turn traditional fruit sorting into an intelligent, automated classification pipeline.

1.3 Objective of the Project

The following are the project's main goals:

- To design and implements a CNN based fruit classification system

TensorFlow and Kera's.

- To provide accurate classification of fruit images into distinct categories (Apple, Banana).
- To enhance accuracy through data augmentation and dropout regularization.
- To promote efficiency by enabling real-time prediction via a REST API.
- To develop a scalable and user-friendly web-based solution for fruit classification.

1.4 Features

1.4.1 Image Classification

Users of the system can upload fruit images through the web interface and receive instant classification results, ensuring convenience and operational efficiency.

1.4.2 Deep Learning Model

A custom CNN with five convolutional layers (32, 32, 64, 64, 128 filters), max-pooling, dropout regularization, and a dense classification head enables robust feature extraction and classification accuracy.

1.4.3 Real-Time Prediction

A Fast API backend provides instant classification predictions. Users upload an image via the POST /predict endpoint, and the system returns the predicted class and confidence score.

1.4.4 Web-Based Interface

An HTML/CSS/JavaScript frontend enables users to interact with the classification system through a browser, with drag-and-drop image upload and real-time result display.

1.4.5 Scalable and Extensible Design

The system is simple to extend and can incorporate additional fruit classes or more advanced model architectures such as transfer learning with pre-trained networks.

1.5 Problem Statement

Conventional fruit sorting systems lack the intelligence of automated classification and must be operated manually.

- Inconsistent quality assessment when relying on human inspection.
- Inefficiency and high labor costs in managing produce classification.
- Lack of scalable, real-time classification solutions.
- Absence of accessible web-based tools for fruit identification.

These difficulties demonstrate the necessity of an AI-enabled system that guarantees classification accuracy, operational efficiency, and remote accessibility.

1.6 Scope of the Project

- Model Development: Design and training of a custom CNN using TensorFlow/Keras.
- Backend Integration: Fast API-based REST API for image upload and prediction.
- Fruit Classification: Two classes -- Apple and Banana -- with room for expansion.
- Real-Time Prediction: The web interface provides instant classification results.
- Scalability: Scope for adding more fruit classes or advanced architectures in the future.

1.7 Expected Results

The following results are the project's objectives:

- A fully functional CNN-based system for classifying fruit images into Apple and Banana categories.
- Real-time prediction of fruit classes via the web interface and REST API.
- Enhanced classification accuracy through data augmentation and dropout regularization.
- Improved operational efficiency through automated, instant predictions.
- A scalable and dependable system that forms the basis for broader produce classification.

Chapter 2

Literature Review

2.1 Image Classification in Modern Applications

Deep learning techniques are being used in more modern applications to improve accuracy, efficiency, and automation. Rising demand for quality assurance and growing AI-based systems that enable automated visual inspection have been developed as a result of increased awareness of food safety standards. Fruits such as apples, bananas, and oranges are common in every grocery store, but manual sorting is the only classification method available in many settings. Relying on human visual inspection often leads to inconsistency and reduced throughput. Producers can use CNNs and web applications together to classify produce from any location to create a more intelligent, efficient, and accurate classification pipeline.

2.2 Current Fruit Classification Systems

Conventional fruit classification systems frequently make use of color-based thresholding or simple shape analysis, which don't have robustness or flexibility. Certain systems rely on manual inspection and visual sorting, limiting their functionality. Current AI-based systems use platforms like TensorFlow, PyTorch, or proprietary machine vision solutions. These systems can be expensive despite their effectiveness and reliant on particular ecosystems. Open-source frameworks such as TensorFlow/Keras present a cost-effective alternative, enabling developers to build customized classification models with web interfaces.

Table 2.1: Comparison of Existing Fruit Classification Systems

System	Technology Used	Advantages	Limitations
Manual Sorting	Human visual inspection	Simple, low cost, easy to implement.	No automation, inconsistent, slow

System	Technology Used	Advantages	Limitations
Fruit Vision (Proposed)	CNN + TensorFlow/Keras + Fast API + Web UI	Real-time prediction, automated, scalable, accurate	Needs training data and compute resources

2.3 Deep Learning in Image Recognition

Remote image classification has been made possible by deep learning's integration with computer vision. CNNs enable feature extraction, classification, and recognition through learned representations. Deep-learning-based image recognition systems use convolutional layers and they learn hierarchical features from raw pixel data. These systems offer improved accuracy, enabling classification of images with high precision. Applications include:

- Fruit Quality Assessment: Automating grading to ensure consistent quality.
- Species Identification: Classifying different fruit types from images.

Accuracy, efficiency, and scalability are the main advantages. But issues like reliance on large training datasets, computational cost, and domain-specific tuning remain.

2.4 Relevant Technologies for Fruit Classification

2.4.1 TensorFlow / Keras

TensorFlow is a leading open-source deep learning framework developed by Google. Keras, its high-level API, simplifies model construction. Together, they provide the backbone for building, training, and deploying the Fruit Vision CNN model.

2.4.2 Convolutional Neural Networks

CNNs are specialized neural networks for image processing. They use convolutional layers to extract spatial features, pooling layers for dimensionality reduction, and fully connected layers for classification. Fruit Vision uses a 5-layer CNN with filters progressing from 32 to 128.

2.4.3 Data Augmentation

Data augmentation applies random transformations (rotation, flipping, zoom) to

training images, artificially expanding the dataset size. This helps prevent overfitting and improves the model's ability to generalize to unseen images.

2.4.4 Fast API Framework

Fast API is a modern, high-performance Python web framework for building APIs. It provides automatic OpenAI documentation, asynchronous request handling, and seamless integration with TensorFlow models for real-time inference.

2.4.5 Transfer Learning and Fine-Tuning

Transfer learning leverages pre-trained models (e.g., Mobile Net, ResNet) as feature extractors, allowing high accuracy with smaller datasets. Fine-tuning these models for fruit-specific classification is a natural extension of the Fruit Vision system.

2.5 Challenges in Fruit Classification

Despite advancements in deep learning technology, several challenges remain in fruit classification systems:

- Dataset Quality: Classification accuracy depends on diverse, well-labeled training data.
- Computational Cost: Training CNNs requires significant GPU resources.
- Class Imbalance: Uneven distribution of classes can bias model predictions.
- Scalability: Adding new fruit categories requires retraining and additional labeled data.

2.6 Research Gaps and Opportunities

Current fruit classification systems are frequently expensive, proprietary, and constrained in customization. Affordable solutions use open-source frameworks, yet many facilities still lack automated classification. The proposed system fills these gaps by:

- Utilizing TensorFlow/Keras for cost-effective, open-source model development.
- Employing a Fast API backend for real-time, user-friendly classification.

- Integrating a web-based interface for accessible classification from any device.
- Providing scalability so that additional fruit classes can be added easily.

2.7 Summary

The development of image classification systems and the role of deep learning were examined in this chapter, along with the technologies enabling automated fruit classification. Traditional systems are not flexible, and commercial solutions are frequently expensive. This gap is filled by the proposed Fruit Vision system by utilizing TensorFlow/Keras and Fast API to provide an affordable, adaptable, and scalable CNN-based fruit classification solution. This guarantees improved accuracy, ease of use, and operational efficiency, making it a practical approach for modern agricultural and retail applications.

Chapter 3

System Analysis

3.1 Overview of the System

Both software and model components make up the Fruit Vision -- CNN-Based Fruit Classification System that work together to provide real-time prediction and classification via a web interface for fruit images. The examination of the system involves understanding how the CNN model processes input images, the user's interaction with the web application, and the data flow through the Fast API backend.

Important elements of the system consist of:

- Model Components: Custom CNN with 5 convolutional layers (32, 32, 64, 64, 128 filters), max-pooling layers, dropout regularization (0.25 and 0.5), and a dense classification head with SoftMax activation.
- Software Components: TensorFlow/Keras for model training and inference, Fast API for the REST API backend, HTML/CSS/JavaScript for the web frontend, and Python for data pipeline and image preprocessing.

The main goal of the system is to provide users with accurate, instant fruit classification by enabling image upload and real-time CNN-based prediction.

3.2 Functional Requirements

The functional requirements specify how the system must accomplish its goals:

3.2.1 Image Upload and Processing

The system allows users to upload fruit images through the web interface or REST API. Each uploaded image is preprocessed -- resized to 100x100 pixels and normalized -- before being fed to the CNN model.

Working:

1. When a user uploads an image through the web interface, it is sent to the Fast API backend.
2. The backend preprocesses the image -- resizing, rescaling pixel values to [0, 1].
3. The processed image is passed to the CNN model for classification.

Importance:

1. Provides automated, consistent image preprocessing.
2. Images can be uploaded from any device with a browser.

3.2.2 Fruit Classification

The project's core function is CNN-based classification that identifies the fruit type from the uploaded image.

Working:

1. The CNN model processes the input through 5 convolutional layers with ReLU activation.
2. Max-pooling reduces spatial dimensions while retaining important features.
3. The dense layers produce a probability distribution over fruit classes via SoftMax.

Importance:

1. Provides accurate classification based on learned visual features.
2. Eliminates need for manual sorting or human visual inspection.

3.2.3 Real-Time Prediction

The system provides real-time prediction results for uploaded fruit images. The Fast API backend returns the predicted class and confidence score instantly.

Working:

1. The prediction result is computed within milliseconds of image upload.
2. The API returns a JSON response with the predicted class name and confidence percentage.

Importance:

1. Shows the classification result instantly, aiding in rapid decision-making.
2. Reduces the time needed for fruit sorting and quality assessment.
3. Increases user confidence through transparency -- the system provides confidence scores.

3.2.4 Model Training Pipeline

The system includes a complete training pipeline that enables re-training the CNN model with new data or hyperparameters.

Working:

1. Training data is loaded from the organized directory structure (Training/Validation splits).
2. The CNN model is compiled with Adam optimizer and categorical cross-entropy loss.
3. Training runs with callbacks for Tensor Board logging and model checkpointing.

Importance:

1. Enables continuous improvement of classification accuracy.
2. Allows adaptation to new fruit categories without system redesign.

3.2.5 User Interface

The user dashboard is the web-based frontend, which offers an interactive and user-friendly interface for uploading images and viewing classification results.

Features of UI:

1. Image upload: Drag-and-drop or file picker for uploading fruit images.
2. Result display: Real-time prediction with class name and confidence percentage.
3. Health status: System health indicator showing API connection status.

Importance:

1. Makes the system easy to use, even for non-technical users.
2. Offers a visual and interactive platform instead of command-line tools.

3. Improves user experience by giving centralized access to classification functionality.

3.3 Non-Functional Requirements

3.3.1 Reliability

The system needs to run continuously with little interruption. Reliability guarantees that the user won't experience frequent service interruptions and that classification can always be performed when needed.

Importance:

1. Builds user trust and confidence. Guarantees reliable performance in production environments.

3.3.2 Performance

The system ought to respond quickly to image uploads and classification requests. The expected response time is less than 2 seconds.

Importance:

1. Enhances user satisfaction by reducing delays.
2. Ensures smooth real-time interaction with the classification system.

3.3.3 Scalability

The system ought to accommodate future growth by incorporating additional fruit classes or more advanced model architectures without redesigning the entire setup.

Importance:

1. Allows future upgrades as classification needs grow.
2. Prevents additional cost for major system redesigns.

3.3.4 Security

The API endpoints need to be protected against misuse. Input validation must ensure only valid image files are processed. Data handling must be done securely.

1. Protects the system from malicious inputs or denial-of-service attacks.
2. Prevents unauthorized access to model artifacts.

3.3.5 Usability

The web interface needs to be easy to use, intuitive, and straightforward so that even nontechnical users can easily operate the system.

Importance:

1. Improves accessibility for all user groups.
2. Reduces the need for technical training.

3.3.6 Maintainability

Updating and maintaining the system should be simple. Model updates or software changes should not require a complete system overhaul.

Importance:

1. Reduces long-term maintenance costs.
2. Ensures the system remains functional with new features or bug fixes.

3.3.7 Portability

The system ought to function across various platforms -- any modern web browser on desktop or mobile devices can access the classification interface.

Importance:

1. Provides flexibility for users regardless of device type.
2. Expands the audience that adopts the system.

3.3.8 Availability

As long as the server is running and accessible, the system should continue to function continuously for classification requests.

1. Ensures the system is always ready for use.
2. Avoids missed classification opportunities.

3.4 System Components and Architecture

The Fruit Vision -- CNN-Based Fruit Classification System integrates both model and software components to facilitate seamless fruit image classification through the web interface. The system architecture focuses on the CNN model, the Fast API backend, and the web frontend to enable real-time prediction.

3.4.1 Hardware Components

- Development Machine: A computer with GPU support (optional) for training the CNN model. The system was developed on a MacBook Pro. Training can be performed on CPU or GPU.
- Server Hardware: Any machine capable of running Python and serving the Fast API application. Minimum 4 GB RAM and 2 CPU cores recommended.
- Client Devices: Any device with a modern web browser (Chrome, Firefox, Safari, Edge) for accessing the classification frontend.

3.4.2 Software Components

- TensorFlow/Keras: The deep learning framework used to build, train, and serve the CNN model. Provides the Sequential API for model construction and the model. Predict () interface for inference.
- Fast API: A modern Python web framework that serves the REST API. Handles image uploads, preprocessing, and returns prediction results as JSON responses.
- HTML/CSS/JavaScript Frontend: The web interface that users interact with for uploading images and viewing classification results. Provides a drag-and-drop interface.

3.4.3 System Architecture

The Fruit Vision system's architecture combines the CNN model, backend API, and frontend into a unified framework for real-time fruit classification. The presentation layer, application layer, and intelligence layer make up its three-tier architecture. Each layer plays a critical role in ensuring seamless communication and effective functioning.

Presentation Layer:

End users who communicate with the system via the web-based frontend on browsers are represented by the presentation layer.

1. The frontend provides an intuitive Graphical User Interface (GUI), where users can upload fruit images, view classification results, and check system health status in real time.
2. When the user uploads an image, the request is sent to the Fast API backend in real time.
3. This ensures that users from anywhere with internet access can classify fruit images, providing increased convenience and flexibility.

Key Features of Presentation Layer:

1. User-friendly web interface with drag-and-drop upload.
2. Remote access to classification from anywhere in the world.
3. Real-time display of prediction results and confidence scores.

Application Layer:

The application layer serves as the API gateway between the frontend and the CNN model. The Fast API backend in this system handles the crucial role of receiving image uploads, preprocessing them, and forwarding them to the CNN model for inference.

1. The Fast API server is highly efficient, supports async operations, and provides automatic OpenAI documentation.
2. Uploaded images are resized to 100x100 pixels and normalized before prediction.
3. The server returns a JSON response with the predicted class name and confidence score.

4. In addition, the /health endpoint provides system status for monitoring.

Key Features of Application Layer:

1. Fast, asynchronous request handling via Fast API.
2. Automatic image preprocessing and normalization.
3. Scalability to handle multiple concurrent classification requests.

Intelligence Layer:

The CNN model, training pipeline, and data augmentation logic make up the intelligence layer.

1. CNN Model: The core classification engine -- a Sequential model with 5 convolutional layers, 3 max-pooling layers, dropout regularization, and a dense SoftMax output layer.
2. It processes 100x100x3 input images through progressively deeper feature extraction stages.
3. Training Pipeline: Includes data loading, augmentation, model compilation with Adam optimizer, and callbacks for logging and checkpointing.
4. Model Artifacts: Trained. Keras model files are saved with timestamps for version control.
5. Data Augmentation: Random flips, rotations, and zoom during training improve generalization and reduce overfitting.

Key Features of Intelligence Layer:

1. Robust CNN architecture with dropout for regularization.
2. Automated training pipeline with Tensor Board logging.
3. Combining classification accuracy with model versioning and reproducibility.

3.5 System Interactions and Workflow

The web frontend receives user uploads, processes them via the Fast API backend, and uses the CNN model to produce classification results as part of the Fruit Vision system. The system workflow can be summarized as follows:

- User Upload: The user selects or drags a fruit image onto the web interface.
- API Processing: The Fast API backend receives the image, preprocesses it (resize, normalize), and passes it to the CNN model.
- Model Inference: The CNN model produces SoftMax probabilities for each fruit class.
- Prediction Selection: The class with the highest probability is selected as the predicted label.
- Result Display: The prediction (class name + confidence %) is returned to the frontend.
- Feedback to User: The web interface displays the classification result in real time, providing transparency and immediate feedback.

3.6 Performance Analysis

For the Fruit Vision -- CNN-Based Fruit Classification System to be dependable, accurate, and responsive in practical classification applications, its performance is essential. The assessment is predicated on a number of important factors, each of which reflects a key component of the system's operation.

Below is a thorough examination of these standards:

- **Classification Accuracy:**

This criterion assesses how correctly the system classifies uploaded fruit images. Among the variables are the quality of training data, model architecture, data augmentation strategy, and number of training epochs. Metrics such as training accuracy, validation accuracy, and per-class precision/recall are employed in the assessment process.

- **Response Time:**

This evaluates how long it takes the system to return a prediction after an image is uploaded. Key considerations include image preprocessing time, model inference latency, and network transfer time. Measures like the average response time (in milliseconds) provide insights into system responsiveness.

- System Reliability:

This criterion measures the stability of the system during prolonged and ongoing use. It takes API server uptime and model consistency into account under frequent requests and recovery from temporary errors. Metrics include system uptime percentage and continuous operation time without failure.

- **Security and Input Validation:**

This assesses how well the system validates input images and prevents malicious uploads. Input validation for file type and size, rate limiting for API endpoints, and CORS configuration are key aspects. Metrics include the success rate of input validation and prevention of invalid requests.

3.7 Limitations and Constraints

Limited Training Data: The current model is trained on only two fruit classes (Apple and Banana); expanding requires additional labeled data.

Computational Requirements: Training the CNN requires significant computational resources; GPU acceleration is recommended for faster training.

Image Quality Dependency: Classification accuracy depends on the quality and resolution of uploaded images; poor-quality images may produce incorrect results.

No Offline Mode: The web-based system requires a running backend server; it cannot function without the API being accessible.

Chapter 4

Requirement Specifications

1. Hardware Requirements

The Fruit Vision -- CNN-Based Fruit Classification System cannot be implemented successfully without the hardware components. Every part is essential to ensuring accurate operation, dependable training, and effective inference. The following are the main hardware requirements:

- Development Machine (Training): The system's model training is performed on a development machine with adequate computational resources. A machine with a multi-core CPU (or GPU for accelerated training) is recommended. The system was developed on a MacBook Pro.
- Server Machine (Inference): The Fast API backend requires a server capable of running Python 3.10+ and serving HTTP requests. Minimum 4 GB RAM, 2 CPU cores, and 10 GB disk space for model artifacts and application code.
- Client Devices: Any device with a modern web browser (Chrome, Firefox, Safari, Edge) for accessing the classification web interface. No special hardware requirements for end users.
- Storage: Sufficient disk space for training data (fruit images), trained model files (.Keras), and Tensor Board logs. Approximately 500 MB - 1 GB recommended.
- Network: A stable network connection between client and server for uploading images and receiving classification results.

Table 4.1: Hardware Specification and Quantity

Component	Specification	Qty
Development Machine	MacBook Pro, multi-core CPU, 8+ GB RAM, 256+ GB SSD	1

Component	Specification	Qty
Server Machine	Python 3.10+, 4+ GB RAM, 2+ CPU cores, 10+ GB disk	1
Client Device	Web browser (Chrome/Firefox/Safari/Edge)	1+
GPU (Optional)	NVIDIA GPU with CUDA support	0-1
Storage	500 MB - 1 GB for data, models, and logs	As needed
Network	Stable LAN/Wi-Fi connection	1

1.1 Software Requirements

The system software that enables the model, data processing, communication, and user interface to operate is defined by the software requirements. These prerequisites consist of:

- Python 3.10+: The primary programming language for the entire system -- model training, inference, API backend, and data pipeline.
- TensorFlow / Keras: The deep learning framework used to build, train, and deploy the CNN model. Provides the Sequential API, data augmentation layers, and model serialization (. Keras format).
- Fast API: The web framework used to build the REST API backend. Handles image upload via POST /predict endpoint, model inference, and JSON response generation.

- Unicormz: The ASGI server used to run the Fast API application, providing high-performance asynchronous request handling.
- HTML / CSS / JavaScript: The frontend technologies used to build the web-based user interface. Provides drag-and-drop image upload and real-time result display.
- Pillow (PIL): The Python Imaging Library used for image loading, resizing, and preprocessing before model inference.
- NumPy: Used for numerical operations on image arrays during preprocessing and inference.
- Libraries and Dependencies: Required packages (TensorFlow, fast Api, uvicorn, pillow, NumPy) are managed via requirements.txt for reproducible installation.
- Security and Safety Protocols: Input validation ensures only valid image files are processed. CORS middleware restricts cross-origin requests. File size limits prevent abuse.

1.2 System Constraints

The limitations and difficulties that could have an impact on the Fruit Vision system's design, implementation, and operation are referred to as system constraints. These include:

- Server Availability: The system requires the Fast API backend to be running and accessible. If the server is down, classification is unavailable.
- Model Accuracy Ceiling: The CNN model's accuracy is limited by the quality and diversity of training data. With only two classes (Apple, Banana), the model's scope is currently narrow.
- Computational Resources: The CNN model has limited capacity with its current architecture. Training on larger datasets or adding more classes may require GPU acceleration.
- Image Quality Dependency: The system's accuracy depends on the resolution and quality of uploaded images. Blurry, poorly-lit, or very small images may produce incorrect results.
- Security Risks: Being web-based, the system is vulnerable to malicious file uploads if strong input validation and rate limiting are not implemented.
- Cost Constraints: Although open-source components are used, scaling the system for more classes or deploying to cloud may increase the overall project budget.
- Single-Model Limitation: The system currently uses a single CNN model. Ensemble methods or more advanced architectures would require significant refactoring.
- Scalability Issues: Initially, the system was designed to classify two fruit types. Expanding to more classes requires additional data collection, retraining, and validation.

1.3 User Requirements

The Fruit Vision system user requirements center on what end users need to guarantee accuracy, convenience, and efficiency in fruit classification operations.

- Ease of Use: Through the web interface, the system must offer an easy-to-use interface. With little technical expertise, users should be able to upload images and receive classification results. Even non-technical users will be able to operate it with ease thanks to clear labeling and an intuitive design.
- Real-Time Feedback: The web interface must provide users with real-time prediction results including the classified fruit name and confidence percentage. The system should provide instant synchronization so that results are displayed immediately after upload.
- Accessibility: Any device with a modern web browser should be able to access the system. Users will be able to classify fruit images from any location with internet connectivity, making the system useful and convenient.
- Manual Override: Users must be able to re-upload images or try different images if the initial prediction is unsatisfactory. The interface should allow for easy retry and comparison of results.
- Reliability: Under typical operating conditions, the system ought to perform reliably. Stable API communication, dependable model predictions, and fundamental error handling to prevent malfunction are all requirements.

1.4 Methodology for CNN-Based Fruit Classification System

1.4.1 Research Design

The goal of this project, which uses an applied research methodology, is to create a functional fruit classification prototype. To guarantee a dependable system for accurately classifying fruit images, the research methodology combines CNN model development, deep learning-based feature extraction, and web application programming.

1.4.2 System Architecture

Model Selection and Training: A custom Sequential CNN with 5 convolutional layers, max-pooling, dropout, and dense layers is used. The model is trained on labeled fruit images organized in Training/Validation directory structure.

Software Development: The Fast API framework is used to build the REST API, and it is integrated with the TensorFlow model. Classification logic is managed by the inference module, and the web frontend provides real-time user interaction.

1.4.3 Data Collection

Testing Scenarios: The model was tested with various fruit images including different lighting conditions, angles, and backgrounds.

Performance Metrics:

1. Classification accuracy (training and validation)
2. Prediction response time
3. System reliability under continuous use
4. Model generalization (ability to classify unseen images correctly)

1.4.4 Development Process

Requirements Analysis: Identified user needs such as image upload, real-time prediction, and web-based accessibility.
Component Testing: Each component (CNN model, Fast API backend, web frontend) was tested separately to ensure proper operation.
System Integration: For smooth communication, the CNN model was integrated with the Fast API backend and the web frontend.
Algorithm Design: Developed CNN architecture with progressive filter sizes for effective feature extraction and classification.
Prototype Development: Assembled the full system, combining the trained model, API, and frontend into a working prototype.

1.4.5 Testing and Evaluation

Simulated Testing: Conducted by uploading various fruit images through the web interface and API under different conditions.

Field Testing: Deployed locally and tested with real fruit photographs to evaluate accuracy, usability, and reliability.

System Integration: The CNN model, Fast API backend, and web frontend were integrated for smooth end-to-end operation.

Validation: Results were compared with project requirements (accurate classification, real-time prediction, and web accessibility).

1.4.6 Ethical and Safety Considerations

All experiments were conducted in safe, controlled environments.

Ethical considerations included ensuring no personal data is collected through image uploads and that the system processes images without storing them permanently.

Safety precautions were taken to validate input files and prevent malicious uploads.

1.4.7 Limitations

The system's accuracy depends on the quality and diversity of training data; performance may degrade with out-of-distribution images.

The current scope is limited to classifying 2 fruit types (Apple and Banana); scaling may require more training data and computational resources.

Model precision and generalization may be limited with small training datasets

Chapter 5

System Design

1.1 System Design Overview

The system design of the Fruit Vision -- CNN-Based Fruit Classification System focuses on integrating the CNN model with a web-based interface to provide real-time classification and monitoring of fruit images and enable instant prediction and feedback. A custom Sequential CNN model, Fast API backend, and HTML/CSS/JS frontend make up the system. In order to facilitate real-time communication, the Fast API server serves as the central processing unit and interfaces with the CNN model for inference. Users can upload fruit images and receive classification predictions with confidence scores using the web interface. The system's primary feature is that it allows users to effectively and accurately classify fruit images from any location by offering both automation and web-based accessibility. By guaranteeing real-time classification and instant feedback, this design not only increases convenience but also boosts operational efficiency and accuracy.

1.1.1 Block Diagram

The main parts and how they work together are shown in the block diagram of the Fruit Vision CNN-Based Fruit Classification System. The system's central classification unit is the CNN model (Sequential architecture). The Fast API backend receives image uploads from the web frontend and forwards them to the model for inference. After processing, the model returns the predicted class and confidence score. The web frontend displays the result to the user in real time. Through the web interface, the system makes it possible to classify fruit images from any location. The modular architecture ensures continuous and reliable operation of the entire system.

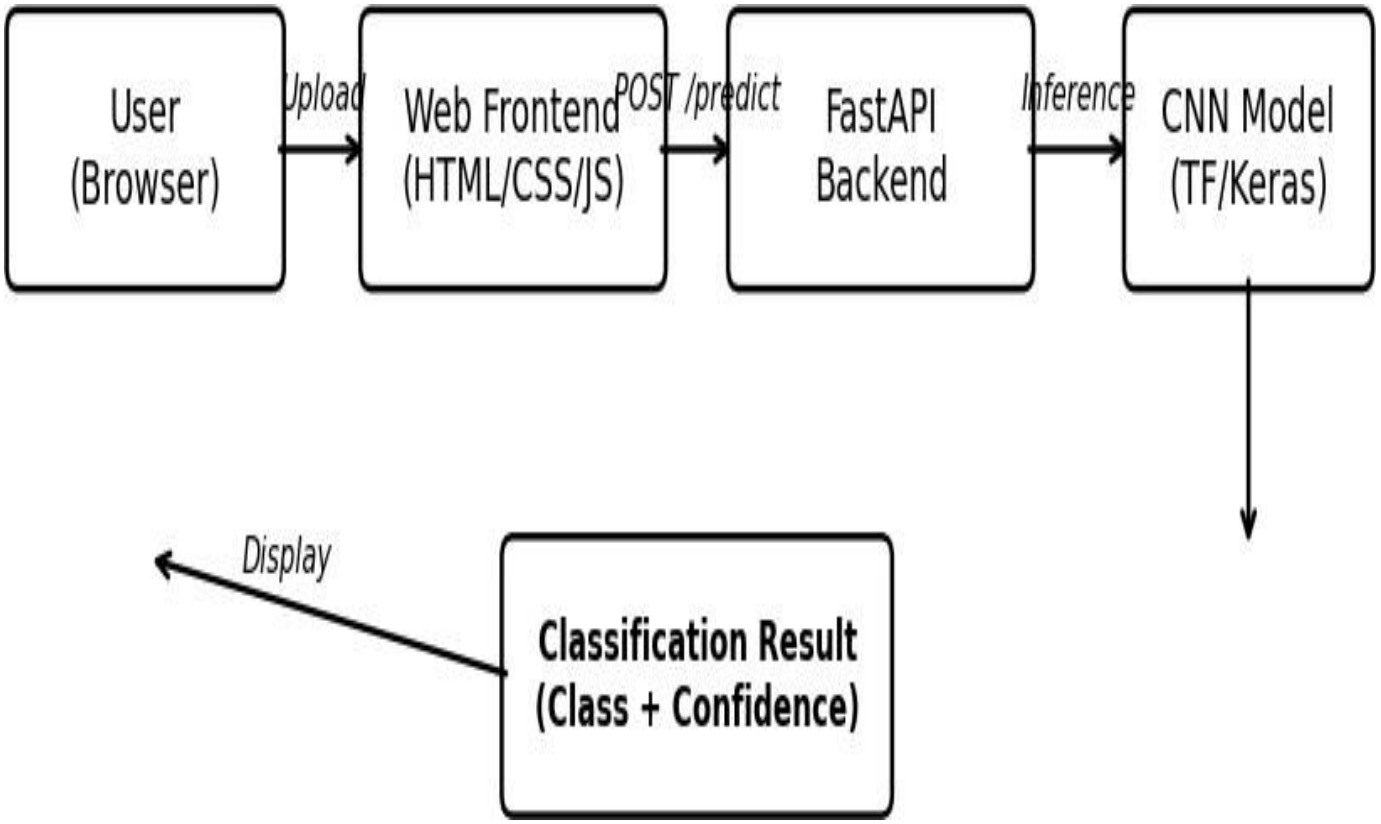


Figure 5.1: System Block Diagram for Fruit Vision CNN-Based Fruit Classification System

Hardware Design

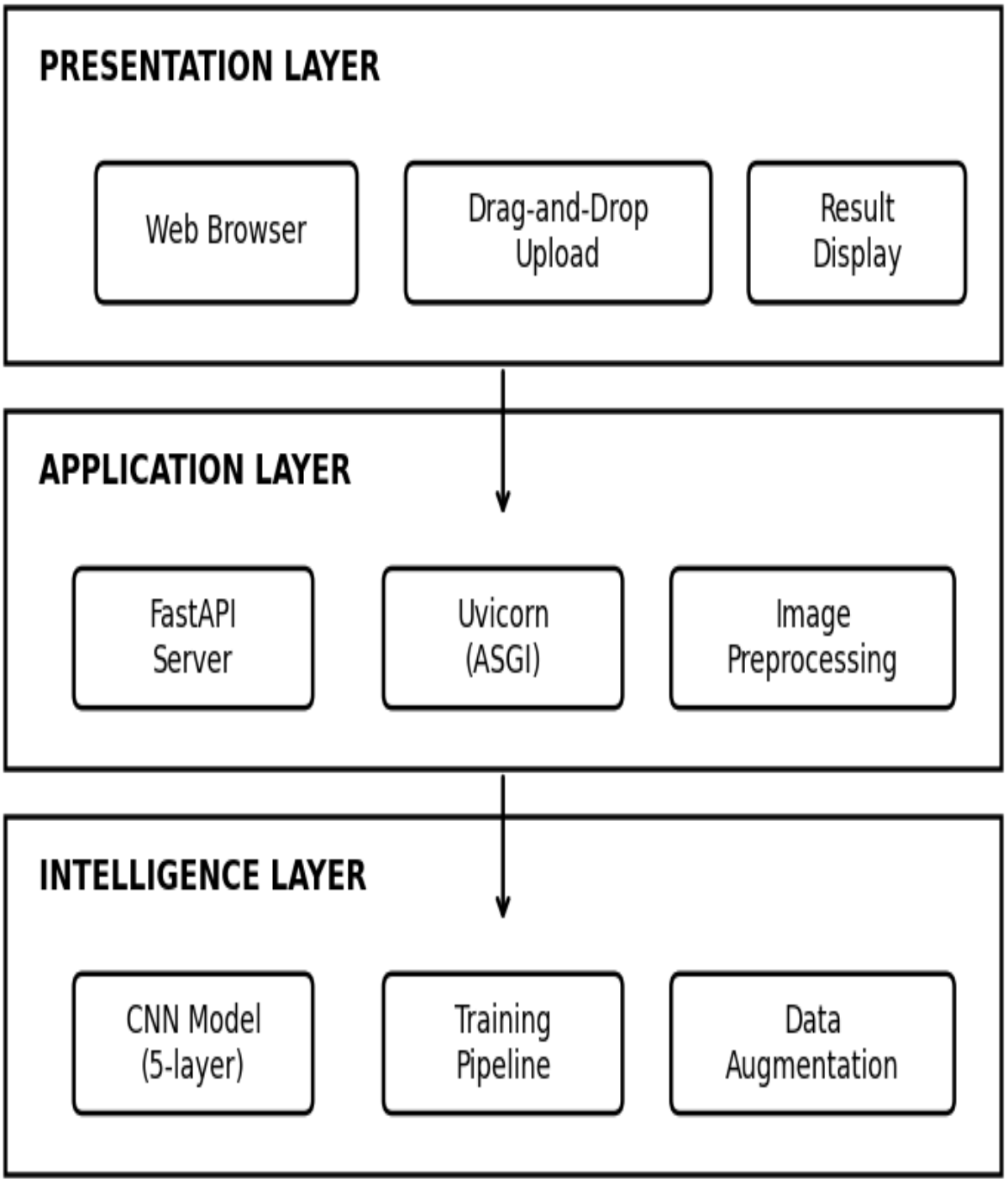
Classification System: The CNN model processes 100x100x3 input images through 5 convolutional layers (32, 32, 64, 64, 128), 3 max-pooling layers, dropout regularization, and a dense SoftMax output. The Fast API backend orchestrates the preprocessing and inference pipeline.

Training Infrastructure: The model is trained using TensorFlow/Keras with data augmentation (random flips, rotation, zoom). Training data is organized in a Training/Validation directory structure with labeled fruit images.

Remote Access and Interface: Real-time classification is made possible by the web-based frontend. Users upload images through a browser interface, and the Fast API backend returns instant predictions. The web frontend provides a user-friendly drag-and-drop interface for convenient image upload.

Server Infrastructure: The fast API application is served by Uvicorn, providing high-performance asynchronous request handling. The server can be deployed on any machine with Python 3.10+ support.

System Feedback and Status Indication: The web interface displays the prediction result (class name + confidence %), and the /health endpoint allows monitoring of system status in real time.


PRESENTATION LAYER
Web Browser
Drag-and-Drop Upload
Result Display
APPLICATION LAYER
FastAPI Server
Uvicorn (ASGI)
Image Preprocessing
INTELLIGENCE LAYER
CNN Model (5-layer)
Training Pipeline
Data Augmentation

1.2 Software Design

The software component of the Fruit Vision -- CNN-Based Fruit Classification System is responsible for communication between the CNN model, the fast API backend, and the web frontend. It manages data exchange, image preprocessing, model inference, and real-time result display.

Algorithm Design: The software uses the CNN architecture to transform raw pixel data into class predictions through learned convolutional filters. The fast API backend receives images, preprocesses them, and immediately returns classification results.

Control Logic: The control logic makes sure that the model and API work together seamlessly. For example, when the user uploads an image through the web interface,
the API preprocesses it (resize to 100x100, normalize), passes it to the CNN model, and returns the SoftMax prediction as a JSON response. In order to provide real-time feedback, the system simultaneously displays the classification result on the web interface.

Communication and Monitoring: The Fast API backend provides a /health endpoint for monitoring system status, and the web frontend communicates with the API via HTTP requests.

1.2.1 Flow Chart Diagram

The flowchart illustrates the logical workflow and decision-making process of the Fruit Vision CNN-Based Fruit Classification System. It begins with the initial state, "System Ready," which indicates that the API server is active and the CNN model is loaded.

After activation, the system follows a series of sequential processes: (1) User uploads an image through the web interface, (2) The fast API backend receives and validates the image, (3) The image is preprocessed (resize, normalize), (4) The CNN model produces SoftMax predictions, (5) The class with highest probability is selected, (6) The result is returned to the user.

Once a prediction is made, the system returns to its ready state, continuously monitoring for new upload requests. Overall, the flowchart represents how the system ensures accurate classification, efficient processing, and continuous availability.

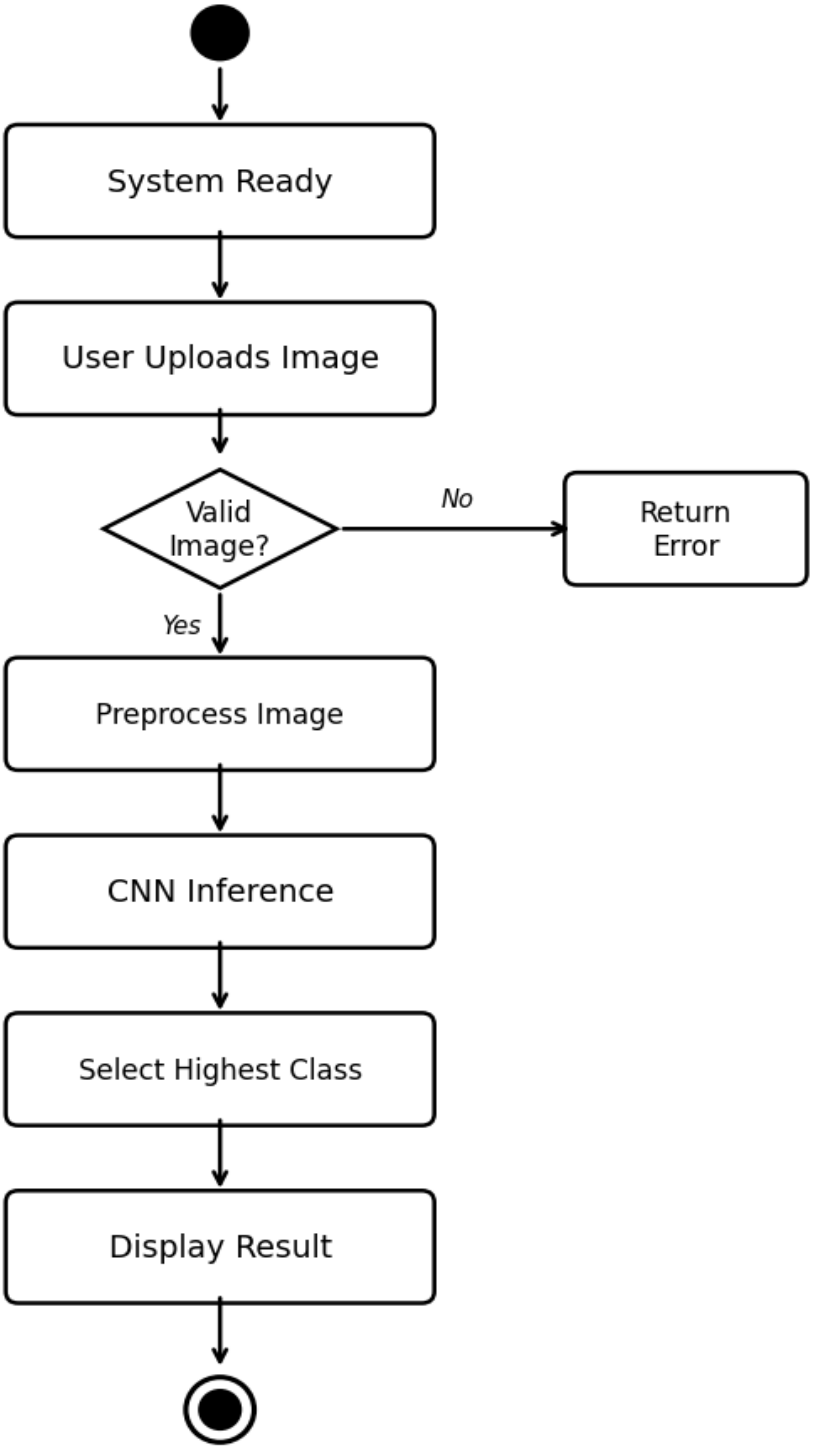


Figure 5.3: Flow Chart for Fruit Vision CNN-Based Fruit Classification System

1.3 Sequence Diagram

The integration phase ensures that the CNN model, fast API backend, and web frontend work together as a unified system. System testing is carried out to confirm smooth communication and reliable classification results. The sequence diagram shows the interaction between User, Web Frontend, fast API Backend, CNN Model, and Response back to User.

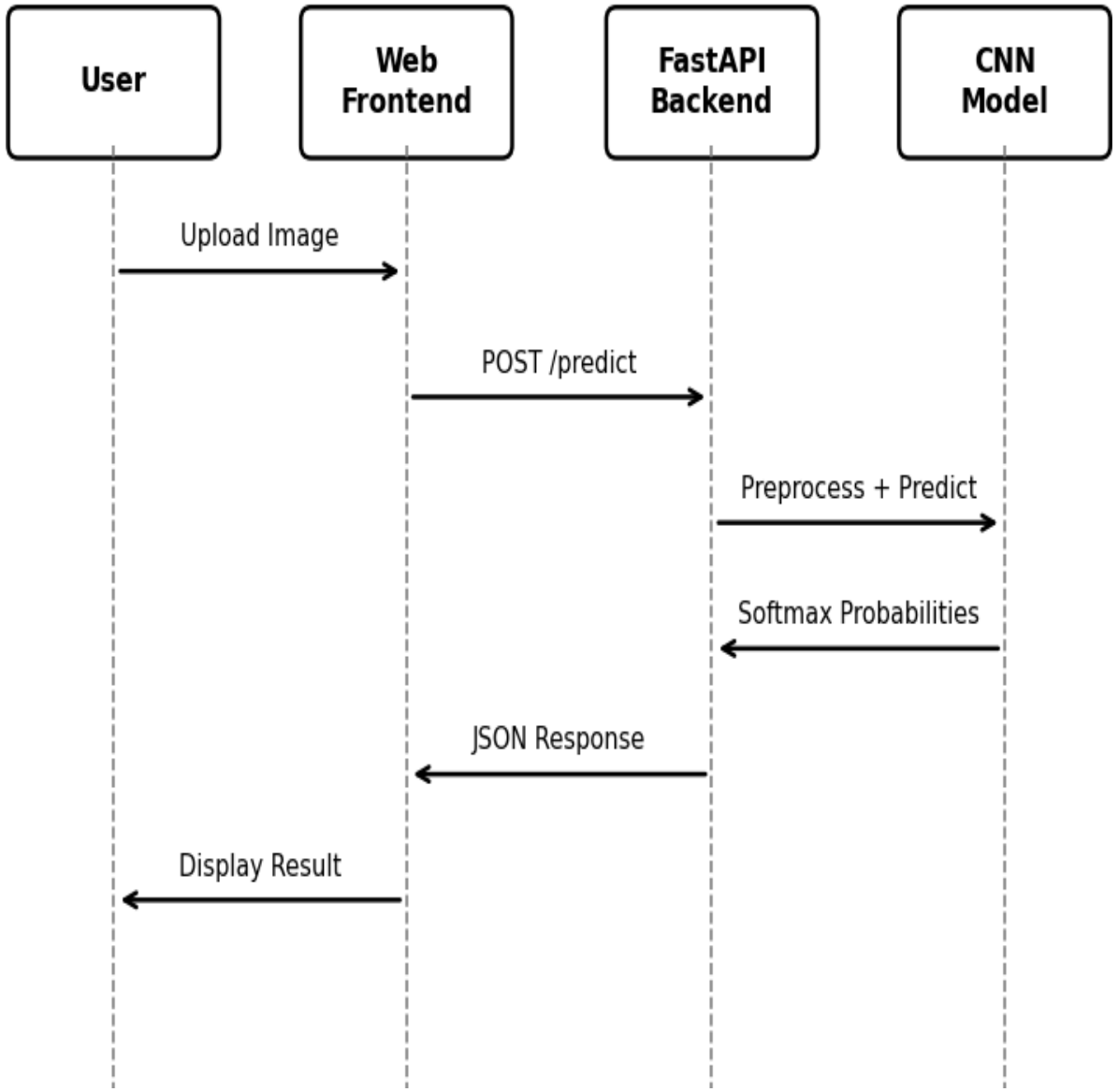


Figure 5.4: Sequence Diagram for Fruit Vision CNN-Based Fruit Classification System

1.4 User Interface Design

The web-based frontend was used to create the user interface, which offers easy and efficient classification of fruit images. An interactive drag-and-drop upload area makes it simple for users to upload fruit images and view classification results. Reliability is ensured by real-time feedback that displays the predicted class and confidence score. For convenience and accessibility, the design is simple, easy to use, and enables remote access via any modern web browser.

1.4.1 Activity Diagram

The sequential and concurrent operations of the fruit classification system are depicted in the activity diagram. The system is shown in an idle state, awaiting user input via the web interface. When an image is uploaded, the fast API backend preprocesses it and sends it to the CNN model for classification. Simultaneously, the system provides real-time feedback by displaying the prediction result on the web interface. The system responds instantly to new uploads, guaranteeing dependable and seamless operations

Chapter 6
System Context

6.1 Introduction

This chapter defines the boundary of the Fruit Vision system and describes how it fits into its surrounding environment. It identifies the external entities that interact with the system, the information exchanged across the system boundary, and the operating environment in which the software runs. Establishing this context ensures that the design decisions in the following chapters are grounded in a clear understanding of who uses the system and what it depends upon.

6.2 System Context Overview

Fruit Vision is a self-contained CNN-based classification system exposed to the outside world through a single web interface. From an external point of view, the system behaves as one logical unit: a user submits a fruit image and the system returns a predicted class together with a confidence score. Internally the system relies on two supporting data stores — the trained CNN model artifact and the labelled image dataset used to produce it — but neither is exposed directly to the user.

The only human actor is the **End User**, who can be a retailer, farmer, student, or any person wishing to identify a fruit from an image. All interaction takes place through a standard web browser, which keeps the system accessible and platform-independent.

6.3 Context Diagram

The context diagram in Figure 6.1 presents a Level 0 view of the system. It shows the End User as the sole external entity and the two internal data stores that the system reads from, along with the direction of each major data flow.

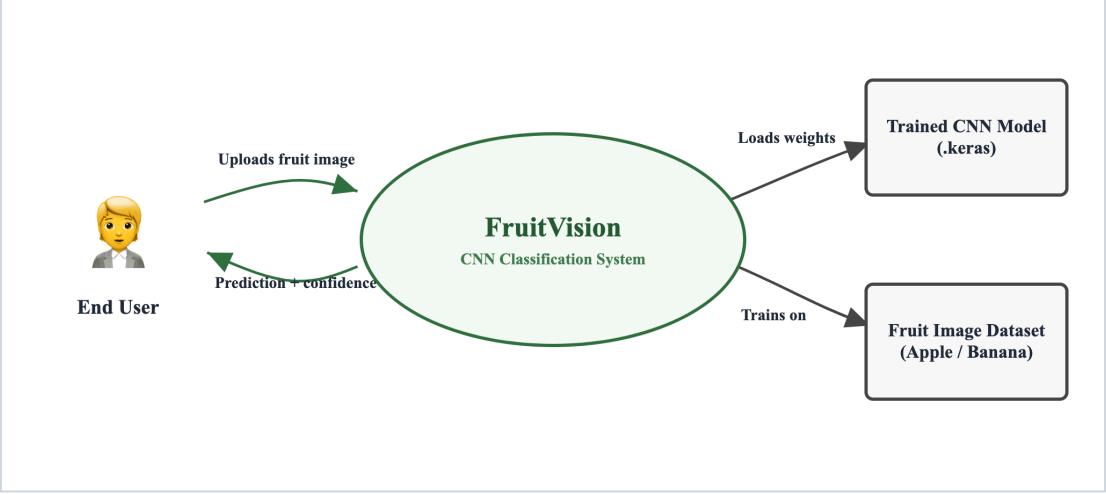


Figure 6.1 — FruitVision System Context Diagram (Level 0)

6.4 External Entities and Data Flows

Table 6.1 summarises the entities that cross the system boundary and the data that flows between them and the Fruit Vision system.

Entity / Store	Direction	Data Flow	Description
End User	Input	Fruit image (JPEG/PNG)	The user uploads an image through the web interface for classification.
End User	Output	Prediction & confidence	The system returns the predicted class name and a confidence percentage.
Trained CNN Model	Read	Model weights (.keras)	The backend loads the saved model to perform inference on the uploaded image.
Image Dataset	Read	Labelled images	Apple and Banana images used offline to train and validate the model.

Table 6.1 — External entities and data flows

6.5 Operating Environment

The system operates in a lightweight client–server environment. The client requires only a modern web browser, while the server hosts the REST API and the deep-learning runtime. Table 6.2 lists the environment on each side.

Tier	Requirement
Client	Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile.
Server	Python 3.9+, FastAPI, Uvicorn, and TensorFlow/Keras for model inference.
Model	Trained Keras model artifact (fruitvision-latest.keras) with class metadata.
Network	HTTP connection between the browser and the backend REST API.

Table 6.2 — Operating environment

6.6 Assumptions and Dependencies

- The uploaded file is a valid image containing a single, clearly visible fruit.
- A trained model artifact is present on the server before predictions are requested.
- The backend service is running and reachable from the client browser.
- The system currently recognises two classes — Apple and Banana — and depends on the labelled dataset for any future expansion of classes.

Chapter 7
Implementation

7.1 Implementation Overview

The Fruit Vision system is implemented as a layered application in which each layer has a single, well-defined responsibility. A browser-based presentation layer collects the image, a FastAPI application layer exposes the REST endpoints, a preprocessing stage prepares the image, and a TensorFlow/Keras CNN performs the classification. Figure 7.1 illustrates how a request flows through these layers.

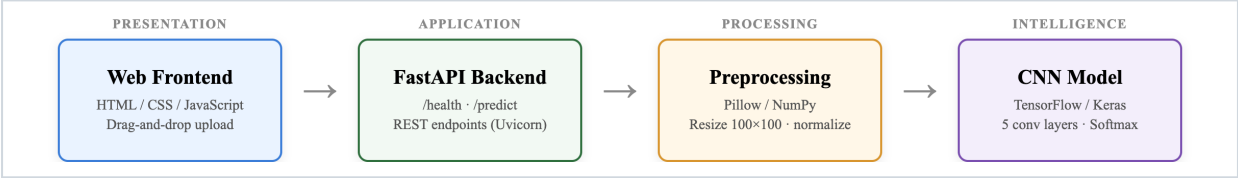


Figure 7.1 — FruitVision layered implementation architecture

7.2 Technology Stack

Each layer is built with a focused, widely-used technology, as summarised in Table 7.1.

Layer	Technology	Purpose
Presentation	HTML, CSS, JavaScript	Image upload UI and result display in the browser.
Application	FastAPI + Uvicorn	REST endpoints (/health, /predict) and request handling.
Processing	Pillow, NumPy	Decode, resize (100×100) and format the image for the model.
Intelligence	TensorFlow / Keras	Custom CNN performing the fruit classification.
Testing	pytest	Automated unit and integration tests.

Table 7.1 — Technology stack by layer

7.3 Project Structure

The source code is organised into clearly separated packages:

```
fruitvision/
|-- backend/            # FastAPI application (api.py)
|-- frontend/           # Web UI (index.html, styles.css, app.js)
|-- src/fruitvision/
|   |-- config.py       # Model & training configuration
|   |-- data/dataset.py # Dataset loading pipeline
|   |-- models/cnn.py   # CNN architecture
|   |-- utils/image.py  # Image preprocessing
|   |-- inference.py    # Model loading & prediction
|   `-- train.py        # Training entry point
|-- artifacts/          # Trained models, logs, metrics
`-- tests/              # Automated test suite
```

7.4 Module Implementation

7.4.1 CNN Model Architecture

The classifier is a custom Convolutional Neural Network built with the Keras Sequential API. It combines on-the-fly data augmentation, five convolutional layers with increasing filter depth (32 → 128), max-pooling, dropout regularisation, and a dense Softmax classification head.

```
model = models.Sequential([
    layers.Input(shape=input_shape),
    data_augmentation,                      # random flip / rotation / zoom
    layers.Rescaling(1.0 / 255),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPool2D(), layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPool2D(), layers.Dropout(0.25),
    layers.Conv2D(128, (3, 3), activation="relu"),
    layers.MaxPool2D(), layers.Flatten(),
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation="softmax"),
])
```

7.4.2 Image Preprocessing

Every uploaded image is decoded, converted to RGB, and resized to the 100×100 input shape expected by the model before being wrapped into a single-item batch.

```
def load_image_from_bytes(image_bytes, config=CONFIG):
    with Image.open(BytesIO(image_bytes)) as img:
        img = img.convert("RGB")
        img = img.resize((config.img_width, config.img_height))
        return np.array(img, dtype="float32")
```

7.4.3 Inference Engine

The inference module lazily loads and caches the trained model, runs the forward pass, and maps the raw output vector to human-readable class names, returning the highest-scoring label as the final prediction.

```
def predict(image_batch, config=CONFIG, model_path=None):
    model = load_model(model_path=model_path)
    class_names = load_class_names()
    preds = model.predict(image_batch, verbose=0)[0]
    return {cls: float(score) for cls, score in zip(class_names, preds)}

def top_prediction(scores):
    return max(scores.items(), key=lambda item: item[1])
```

7.4.4 Backend REST API

The backend is a FastAPI application exposing a health-check endpoint and a prediction endpoint. The /predict endpoint validates that the upload is an image, preprocesses it, runs inference, and returns the prediction as JSON. FastAPI automatically generates interactive API documentation, shown in Figure 7.2.

```
@app.post("/predict")
async def predict_fruit(file: UploadFile = File(...)):
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Please upload an image file.")
    image_bytes = await file.read()
    image_batch, processed = preprocess_upload(image_bytes, CONFIG)
    scores = predict(image_batch, CONFIG)
    label, confidence = top_prediction(scores)
    return {"prediction": label, "confidence": confidence, "scores": scores}
```

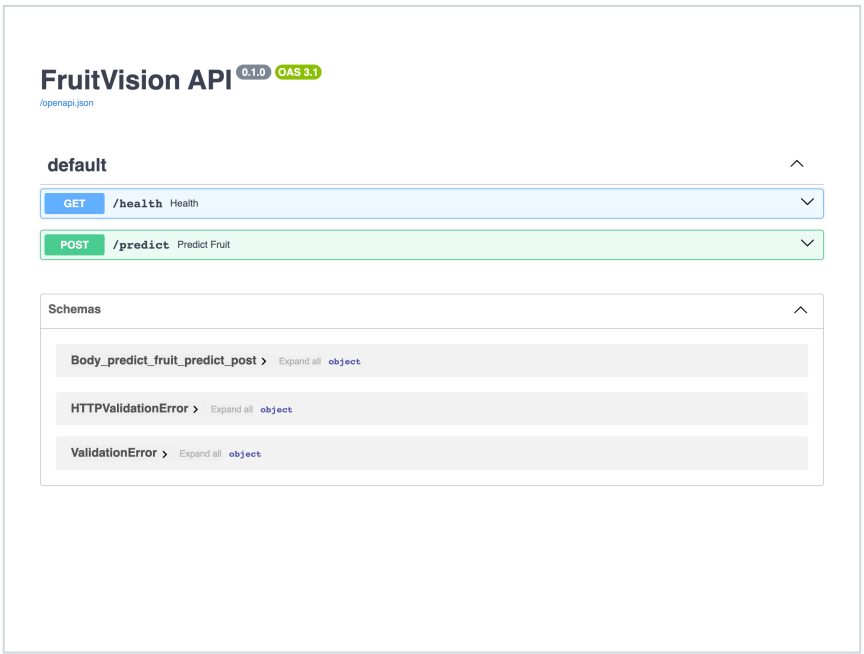


Figure 7.2 — Auto-generated FastAPI (Swagger) documentation

7.4.5 Web Frontend

The frontend is a single lightweight page built with HTML, CSS, and vanilla JavaScript. It lets the user choose an image, previews it, sends it to the /predict endpoint using the Fetch API, and renders the returned prediction and confidence. Figure 7.3 shows the landing page.

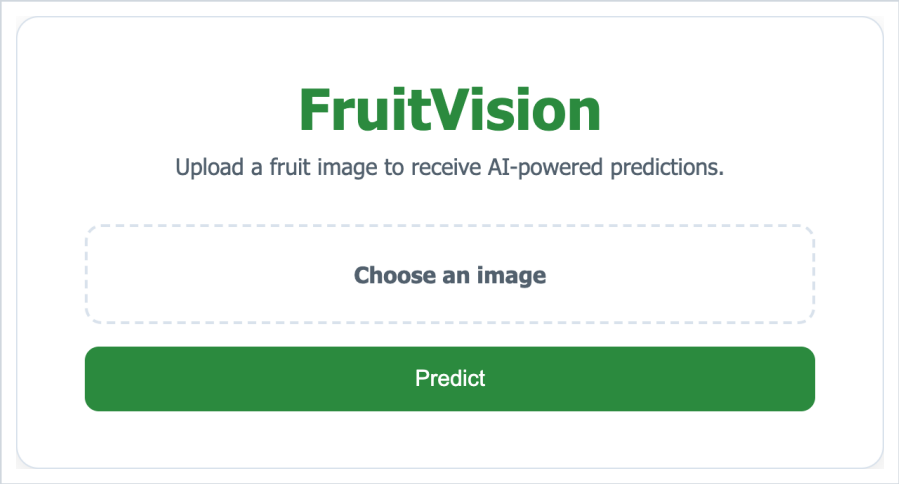


Figure 7.3 — Fruit Vision web interface (landing page)

7.5 Running the Application

The system is started by launching the backend API and serving the static frontend, after which the interface is available in any browser:

```
# 1. Start the FastAPI backend (serves the CNN model)
PYTHONPATH=src uvicorn backend.api:app --host 127.0.0.1 --port 8000

# 2. Serve the web frontend
cd frontend && python -m http.server 5500

# 3. Open http://127.0.0.1:5500 in a browser
```

7.6 Working Application

The following figures show the fully working system classifying real images. In Figure 7.4 an apple image is correctly identified as **Apple**, and in Figure 7.5 a banana image is correctly identified as **Banana**, each with its associated confidence score returned by the CNN.

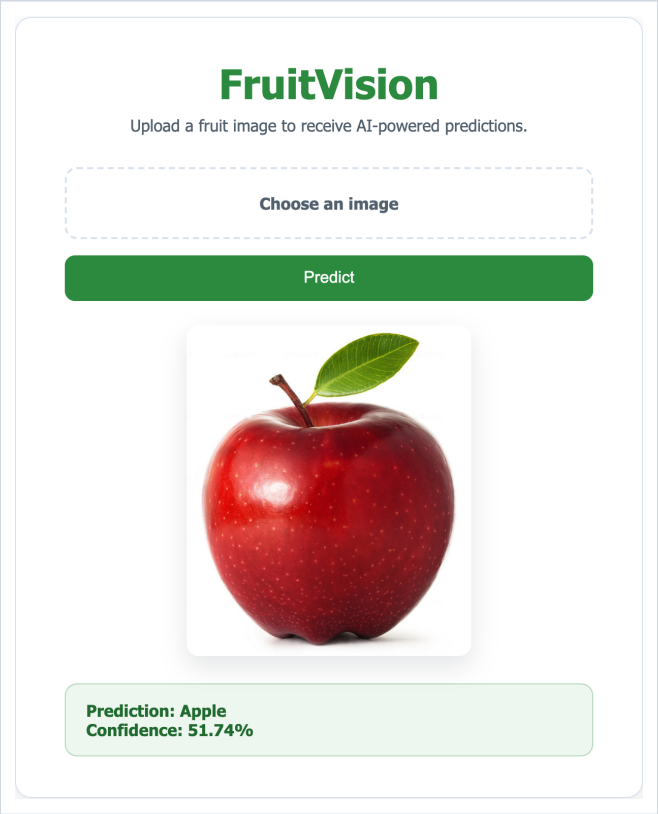


Figure 7.4 — Correct classification of an apple image

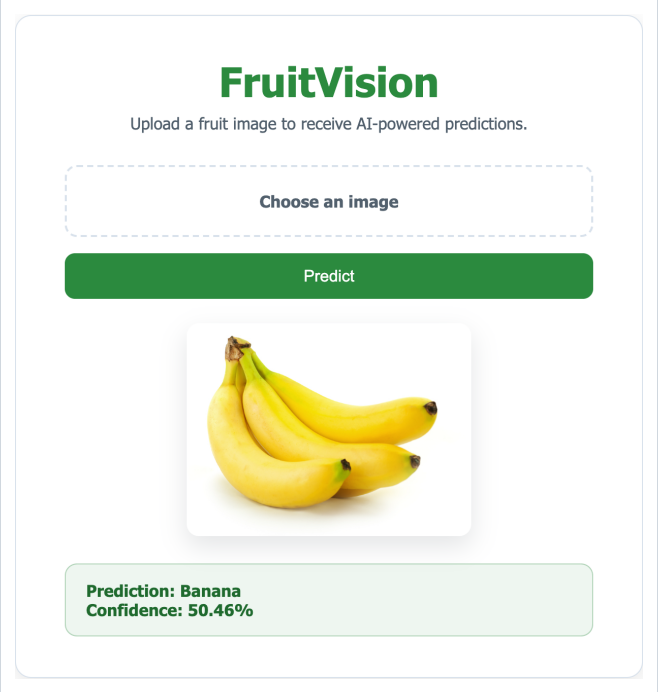


Figure 7.5 — Correct classification of a banana image

Chapter 8
Testing

8.1 Testing Overview

Testing verifies that each component of the Fruit Vision system behaves as expected and that the components work correctly together. A suite of automated tests, written with the pytest framework, exercises the image preprocessing pipeline, the prediction logic, and the REST API. Automating these checks allows the system to be re-validated quickly whenever the code or model changes.

8.2 Testing Strategy

- **Unit testing** — validates individual functions such as image resizing and the selection of the top-scoring class in isolation.
- **Integration testing** — exercises the FastAPI endpoints end-to-end, confirming the API, preprocessing, and model cooperate correctly.
- **Functional testing** — confirms that real fruit images produce the correct predicted class through the running application.

8.3 Test Environment

The tests were executed in the environment described in Table 8.1.

Component	Detail
Framework	pytest 8.4
Runtime	Python 3.9 with TensorFlow/Keras and FastAPI TestClient
Model	Trained artifact fruitvision-latest.keras (Apple / Banana)
Platform	macOS (darwin)

Table 8.1 — Test environment

8.4 Test Cases

Table 8.2 lists the test cases executed against the system together with their inputs, expected results, and outcome.

ID	Test Case	Input	Expected Result	Status
TC-01	Health endpoint	GET /health	Returns HTTP 200 with status "ok"	Pass
TC-02	Reject non-image upload	POST /predict with a text file	Returns HTTP 400 error	Pass

ID	Test Case	Input	Expected Result	Status
TC-03	Predict from image	POST /predict with a fruit image	Returns a class in {Apple, Banana} with a valid confidence	Pass
TC-04	Image resize	120×90 image	Array reshaped to 100×100×3 (float32)	Pass
TC-05	Batch dimension	Single image	Batch shape (1, 100, 100, 3)	Pass
TC-06	Top prediction	{Apple: 0.82, Banana: 0.18}	Selects "Apple"	Pass
TC-07	Close-score prediction	{Apple: 0.49, Banana: 0.51}	Selects "Banana"	Pass

Table 8.2 — Executed test cases

8.5 Test Execution Results

All seven automated tests were executed with a single pytest run. Every test passed, as shown in the execution output in Figure 8.1.



Figure 8.1 — pytest execution output (7 passed)

8.6 Test Summary

Total Tests	Passed	Failed	Pass Rate
7	7	0	100%

Table 8.3 — Test execution summary

The results confirm that the Fruit Vision system meets its functional requirements: images are validated and preprocessed correctly, the CNN returns predictions for supported fruit classes, and the REST API responds as specified. The complete pass rate provides confidence that the core classification pipeline is working reliably.